tracey — Component-Level Design

<course / module name>

<group name>
<member 1>
<member 2>
<member 3>

<date>

Abstract (updated)

tracey is a visual degree-plan builder for Krea University (SIAS). A student picks a programme — single major, double major, major + minor, or major + minor + concentration — and tracey lays out a recommended three- or four-year plan on a year × trimester grid. Courses are dragged between a searchable catalogue and the grid; on every change the plan is re-checked against the university's real rules (credit floors and ceilings, prerequisite ordering, which trimester and which year a course is actually offered in, the foundation-core placement rules, and the audit allowance) and every problem is surfaced as a plain-language hint rather than a silent failure.

The catalogue, the per-programme requirements and the offering data are *scraped* from the university's own programme pages and term timetables rather than hand-typed, so the plan reflects what is genuinely on offer. The application runs entirely in the browser; a student's plan is autosaved locally and never leaves their machine.

1. Software components

The system splits along one clean seam: an *offline data pipeline* that runs before the build and produces static JSON, and a *client-side application* that consumes it. There is no server and no database at runtime.

Build time — offline, stdlib-only Python

Component	**Responsibility**	**Realised as**
Scraper	Fetches and parses the university's programme pages (structure tabs, graduation-requirement credit tables); caches raw HTML.	`scrape_programmes.py`
Catalogue generator	Turns seven term timetables (CSV) into the catalogue: credits, terms offered, eligible years, faculty, and the prerequisite graph.	`generate.py`
Requirements builder	Joins the scrape against the catalogue, resolving website course titles to codes; emits per-programme required sets, choice pools and electives.	`build_requirements.py`
Bundle slimmer	Strips the generated JSON down to the fields the app reads, and splits course descriptions into a lazily fetched chunk.	Vite plug-in

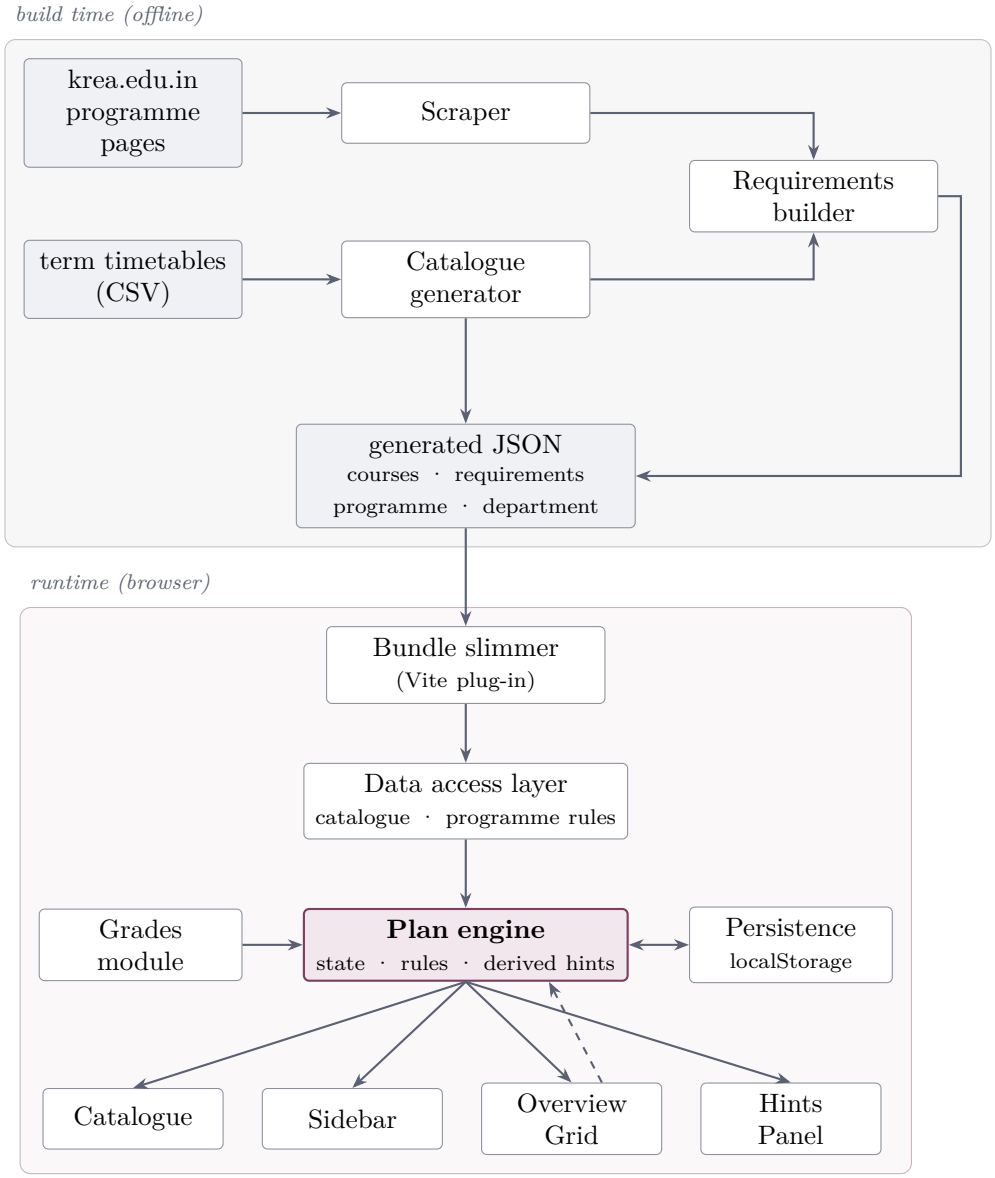
Runtime — browser, SvelteKit + TypeScript

Component	Responsibility	Realised as
Data access layer	Typed reads over the generated JSON plus the programme logic: required/choice/elective sets, substitutes, reachability, recommended pathways.	`data.ts`
Plan engine	*The core.* Holds all plan state and derives every consequence of it: placements, credit totals, the rule checks, the hint list. Single source of truth.	`plan.svelte.ts`
Grades module	Grade scale, weighted TGPA/CGPA, and the "what do I still need" recovery calculation.	`grades.ts`
Persistence	Autosave of the working plan, named saved plans, prerequisite overrides, and display settings — all in `localStorage`.	`savedPlans`, `settings`
Presentation	The four-column shell and its panels (below), each a pure view over the engine.	Svelte components
Admin tools	Separate route for inspecting the catalogue and editing the prerequisite graph; exports edits as JSON.	`/admin`

The presentation layer decomposes into six panels:

- **Sidebar** — programme pickers (degree length, structure, major/minor).
- **Catalogue** — search and subject filters over the course list; emits drag sources.
- **OverviewGrid** / **TrimesterZoom** — the year × trimester board and its single-trimester zoom, drawn at true credit footprint.
- **HintsPanel** — statistics, choice pools, and the live rule hints.
- **DetailsPanel** — metadata for the selected course.
- **GradesPanel** — grade entry and GPA read-out.

2. Architecture



Solid arrows carry derived state *down* to the views; the dashed arrow is a user event calling back *up* into the engine; the double arrow is read/write persistence. Note that *no* view component talks to another view component — every interaction goes through the Plan engine.

3. How the components interact

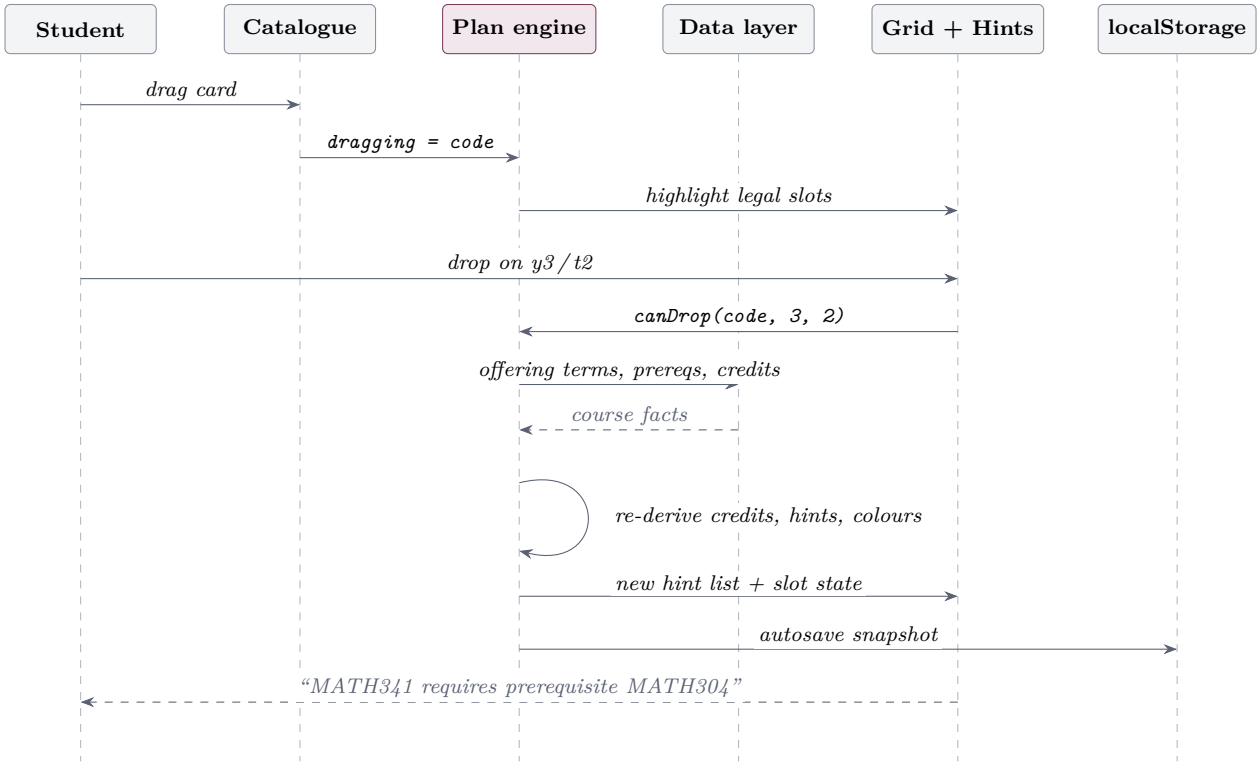
The architecture is deliberately a *star, not a chain.* One reactive `Plan` instance is created by the application shell and passed to every panel as a property. Panels never hold plan state of their own and never speak to each other:

1. **Views call methods, they do not mutate.** A drag onto the grid calls `plan.place(code, year, trim)`; a programme change calls the picker's setter. All writes funnel through the engine.
2. **Everything else is derived.** Credit totals, course colours, the filtered catalogue, statistics and the hint list are computed *from* plan state, so two panels can never disagree — there is no cache to invalidate.

3. **Rules live in exactly one place.** Every check (12-credit floor, 16/20-credit load, prerequisite order, offering term, year-1 core-only rule, audit allowance, total-credit ceiling) is evaluated inside the engine and emitted as a typed hint; the HintsPanel merely renders what it is given.
4. **Persistence is a side effect, not a step.** An effect watching plan state serialises a snapshot to `localStorage`, keeping a backup of the previous value. Restoring re-validates the snapshot before adopting it, so a stale or corrupt save can never load a broken plan.
5. **The pipeline is upstream and one-way.** The Python components run before the build and never at runtime; the application only ever *reads* their output. The website remains the source of truth for what a programme requires.

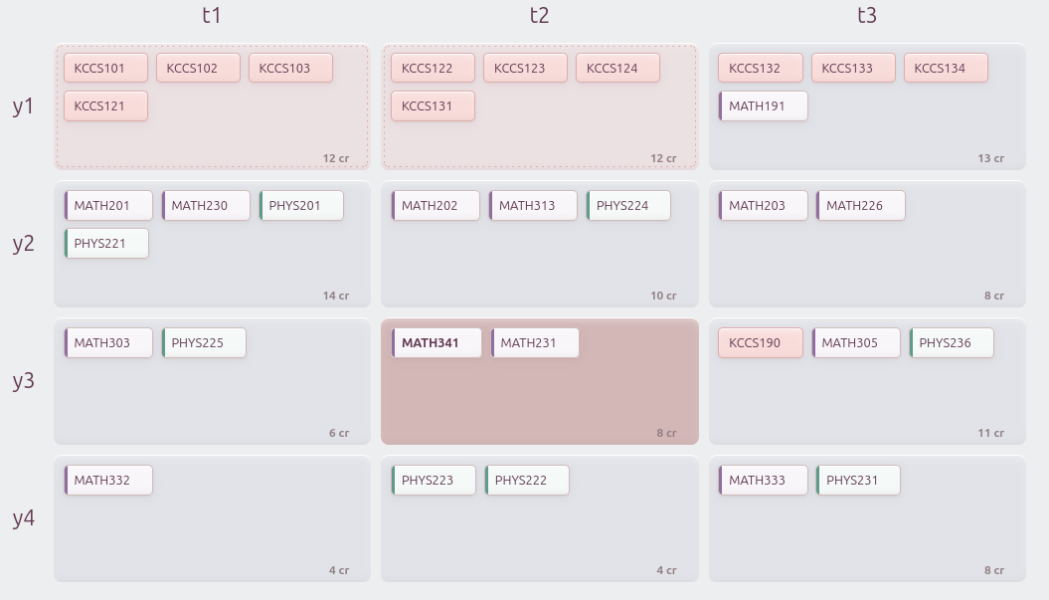
4. Sequence: placing a course on the plan

The interaction below is the system's hot path — the student drags a course card from the catalogue onto a trimester slot.

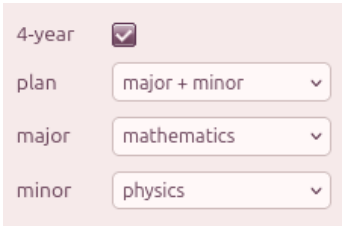


The illegal placement is *not* rejected. The engine accepts the drop and reports the consequence, because a half-built plan that the student is still working on is a normal state, not an error — the hint stays until it is fixed or explicitly waived.

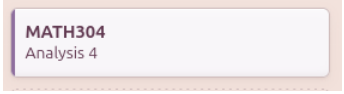
5. The components on screen



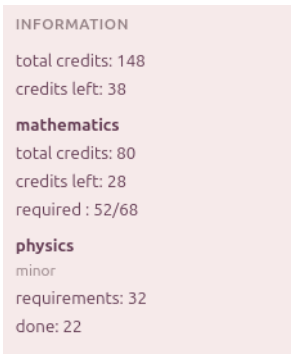
OverviewGrid — the year × trimester board: credit totals per slot, foundation-core courses in their own colour, year-1 slots marked core-only, and the offending trimester tinted where a hint applies.



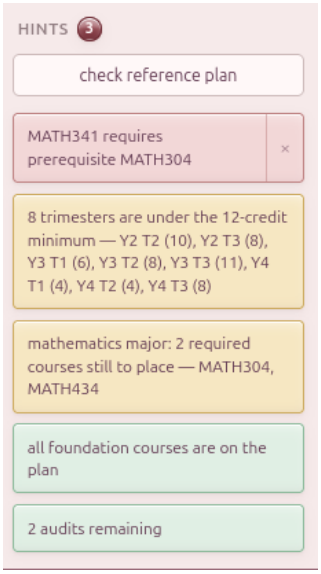
Sidebar



CatalogueCard



HintsPanel — statistics



HintsPanel — live hints

Every number and every hint in the two right-hand panels is derived from the same `Plan` instance that draws the grid — which is why the "2 required courses still to place" count, the prerequisite hint, and the tinted slot on the board can never drift apart.